

**Universidad de las Américas Puebla (UDLAP)**

**Licenciatura en Ciencia de Datos**

# Actividad 14: Paso de mensajes con UDP y TCP

Andrea Hernandez Huerta

ID: 175523

Heriberto Espino Montelongo

ID: 175199

Profesor: Dr. Victor Giovanni Morales Murillo

Curso: Programación Concurrente

Fecha de entrega: Por definir

# Índice general


**1. Análisis del paso de mensajes** **1**

1.1. Marco del problema . . . . . 1

1.2. Análisis de UDP . . . . . 2

1.3. Análisis de TCP . . . . . 3

**2. Servidor y cliente UDP** **4**

2.1. Entradas y procesamiento . . . . . 4

2.2. Salidas, control y conceptos . . . . . 4

2.3. Diagrama de intercambio UDP . . . . . 5

2.4. Evidencia de código y terminales . . . . . 5

**3. Directorio de servicios TCP** **7**

3.1. Entradas y procesamiento . . . . . 7

3.2. Ejemplo esperado y salida observada . . . . . 7

3.3. Diagrama de intercambio TCP . . . . . 8

3.4. Evidencia de código y terminales . . . . . 8

**4. Pruebas y comparación** **10**

4.1. Seis ejecuciones reproducibles . . . . . 10

4.2. Preguntas de análisis . . . . . 10

4.3. Conclusión . . . . . 11

# Índice de figuras

2.1. Código Java del cliente y servidor UDP; captura real pendiente. 5
2.2. Terminal del servidor UDP; captura real pendiente. . . . . 5
2.3. Terminal del cliente UDP; captura real pendiente. . . . . 6

3.1. Código Java del cliente y servidor TCP; captura real pendiente. 8
3.2. Terminal del servidor TCP; captura real pendiente. . . . . 8
3.3. Terminal del cliente TCP; captura real pendiente. . . . . 9

# Índice de cuadros

1.1. Elementos de la implementación UDP. . . . . 2
1.2. Elementos de la implementación TCP. . . . . 3

3.1. Solicitudes del directorio y respuestas. . . . . 7

4.1. Resultados de las seis sesiones cliente-servidor. . . . . 10

# 1. Análisis del paso de mensajes

**Instrucción.** Identificar emisor, receptor, mensaje, canal, operaciones, direccionamiento, bloqueo y tipo de comunicación de cada implementación.

## 1.1 Marco del problema

Los programas de esta actividad son **procesos independientes**: no utilizan variables compartidas para intercambiar información. Se comunican enviando bytes a través de *sockets* de red sobre la interfaz local (`localhost`). Un mensaje tiene un emisor, un receptor y un canal que lo transporta. El cliente solicita una operación y el servidor envía una respuesta.

## 1.2 Análisis de UDP

**Cuadro 1.1:** Elementos de la implementación UDP.

| **Elemento** | **Identificación** |
|---|---|
| Emisor y receptor | El cliente envía la solicitud; el servidor la recibe y después responde al cliente. |
| Mensaje | Texto UTF-8; la respuesta es `Mensaje recibido: [mensaje]`. |
| Canal | Datagramas UDP mediante `DatagramSocket`, puerto 5000. |
| Envío y recepción | `send()` y `receive()` con `DatagramPacket`. |
| Direccionamiento | Directo: `localhost:5000`. La respuesta se envía a la IP y puerto de origen del datagrama recibido. |
| Bloqueo | `receive()` espera un datagrama; la lectura del teclado espera entrada. `send()` no espera una confirmación de aplicación. |
| Comunicación | UDP no establece conexión ni garantiza entrega; *nuestro diálogo* es síncrono, pues cada envío del cliente va seguido de una espera de respuesta. |

## 1.3 Análisis de TCP

**Cuadro 1.2:** Elementos de la implementación TCP.

| Elemento | Identificación |
|---|---|
| Emisor y receptor | El cliente envía órdenes; el servidor responde con el resultado de cada orden. |
| Mensaje | Líneas de texto con `REGISTRAR`, `BUSCAR` o `SALIR`. |
| Canal | Conexión TCP creada con `Socket` y `ServerSocket`, puerto 6000. |
| Envío y recepción | `println()` escribe una línea; `readLine()` lee la siguiente. |
| Direccionamiento | Directo: `localhost:6000`; el sistema asigna un puerto temporal al cliente. |
| Bloqueo | `accept()` espera conexiones; `readLine()` espera una línea o cierre; conectar puede esperar o fallar. |
| Comunicación | Solicitud y respuesta síncronas al nivel de aplicación: el cliente espera el resultado antes de enviar otra orden. TCP ofrece un flujo fiable y ordenado de bytes. |

# 2. Servidor y cliente UDP

**Instrucción.** Crear un servidor en el puerto 5000 que reciba datagramas, identifique dirección y puerto de origen, confirme su recepción y continúe hasta recibir `SALIR`; el cliente lee mensajes del teclado, los envía a `localhost` y muestra las respuestas.

## 2.1 Entradas y procesamiento

El usuario introduce una línea de texto, por ejemplo `Hola servidor`. El `ClienteUDP` la convierte a UTF-8 y crea un `DatagramPacket` con destino al puerto 5000. `ServidorUDP` permanece en `receive()` hasta obtener un datagrama; convierte sus bytes de nuevo a texto, muestra la dirección IP y el puerto del cliente y responde `Mensaje recibido: Hola servidor`. El cliente ejecuta también `receive()` para esperar dicha respuesta. Al enviar `SALIR`, el servidor confirma el mensaje y termina.

## 2.2 Salidas, control y conceptos

Las estructuras de control principales son `while`, `if` y el cierre automático de sockets mediante `try-with-resources`. El cliente tiene un tiempo máximo de espera de tres segundos por respuesta para evitar una espera ilimitada cuando no existe servidor. Esta limitación no convierte a UDP en un protocolo fiable.

## 2.3 Diagrama de intercambio UDP

| Cliente UDP | | Servidor UDP |
|---|---|---|
| `send(datagrama)` | ⟶ | `receive()` |
| `receive()` | ⟵ | `send(confirmación)` |
| `SALIR` | ⟶ | Recibe y responde |
| Finaliza | ⟵ | Confirma y finaliza |

## 2.4 Evidencia de código y terminales

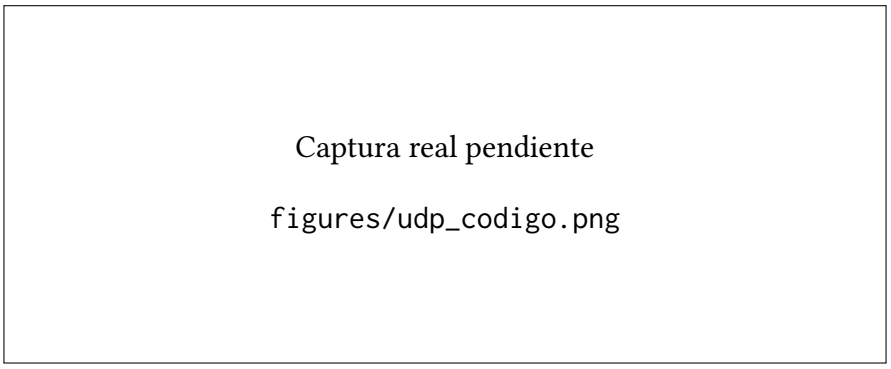



**Figura 2.1:** Código Java del cliente y servidor UDP; captura real pendiente.

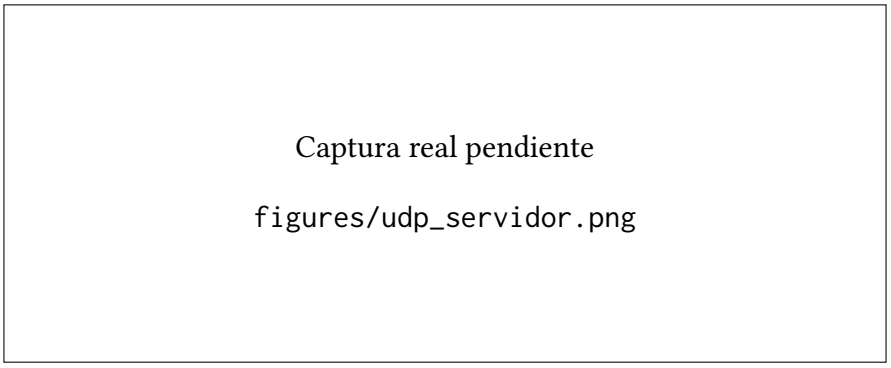



**Figura 2.2:** Terminal del servidor UDP; captura real pendiente.

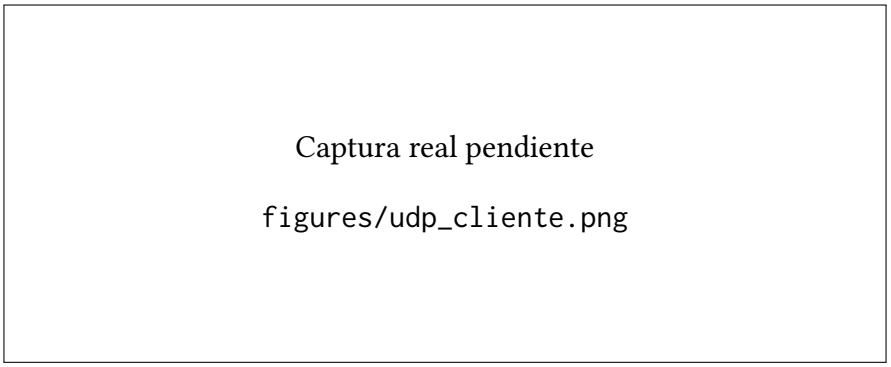



**Figura 2.3:** Terminal del cliente UDP; captura real pendiente.

# 3. Directorio de servicios TCP

**Instrucción.** Implementar un servidor TCP con `ServerSocket(6000)` y un directorio `HashMap` que admita `REGISTRAR nombre puerto`, `BUSCAR nombre` y `SALIR`; construir un cliente interactivo para enviar las solicitudes y mostrar las respuestas.

## 3.1 Entradas y procesamiento

El servidor abre el puerto 6000 y utiliza `accept()` para aceptar un cliente. Conserva los nombres y puertos de servicios en un `HashMap<String,Integer>`. Por cada línea del cliente, separa sus palabras y ejecuta la acción indicada. Se valida que los puertos registrados sean números entre 1 y 65535. Una petición de búsqueda de un nombre ausente devuelve `Servicio no encontrado`. La orden `SALIR` devuelve `Conexion cerrada`, cierra la conexión y termina el servidor.

## 3.2 Ejemplo esperado y salida observada

**Cuadro 3.1:** Solicitudes del directorio y respuestas.

| Solicitud | Respuesta |
|---|---|
| `REGISTRAR calculadora 7000` | Servicio registrado |
| `BUSCAR calculadora` | Servicio encontrado en el puerto 7000 |
| `BUSCAR inventario` | Servicio no encontrado |
| `SALIR` | Conexion cerrada |

El programa utiliza una estructura `HashMap`, ciclos `while`, decisiones `if/else`, lectura por líneas y un `try-with-resources`. Procesa una conexión a la vez, suficiente para el ejercicio. Los servicios se conservan en memoria mientras el servidor siga ejecutándose.

## 3.3 Diagrama de intercambio TCP

| Cliente TCP | | Servidor TCP |
|---|---|---|
| `new Socket(...)` | ⟶ | `accept()` |
| `REGISTRAR ...` | ⟶ | Guarda servicio |
| Muestra respuesta | ⟵ | `Servicio registrado` |
| `BUSCAR ...` | ⟶ | Consulta directorio |
| Muestra respuesta | ⟵ | Resultado de la búsqueda |
| `SALIR` | ⟶ | Confirma y cierra |

## 3.4 Evidencia de código y terminales

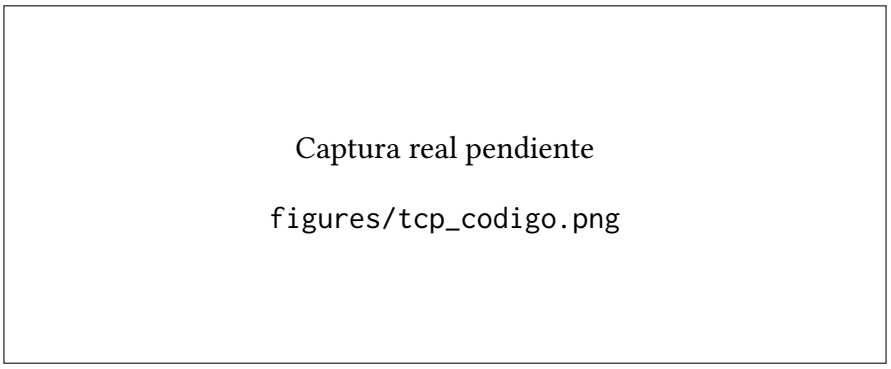



**Figura 3.1:** Código Java del cliente y servidor TCP; captura real pendiente.

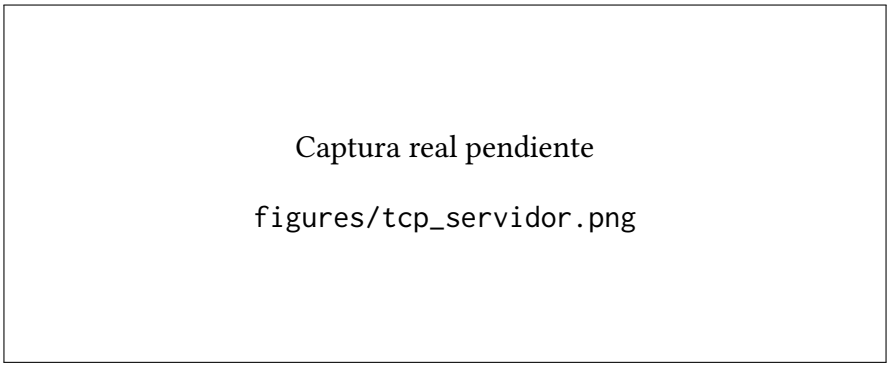



**Figura 3.2:** Terminal del servidor TCP; captura real pendiente.

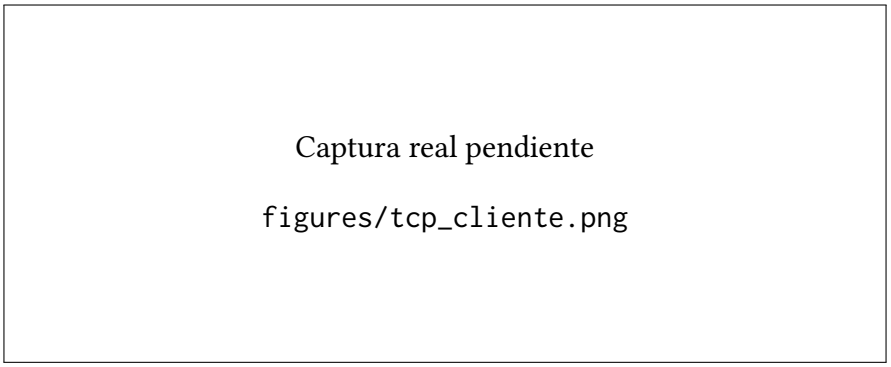



**Figura 3.3:** Terminal del cliente TCP; captura real pendiente.

# 4. Pruebas y comparación

## 4.1 Seis ejecuciones reproducibles

Se ejecutaron **tres sesiones completas de cada implementación**, iniciando un proceso servidor y otro cliente en cada prueba. Los resultados siguientes se obtuvieron con el script `experiment001/run_tests.py`, que almacena las salidas reales de ambas terminales en archivos de texto y verifica automáticamente las respuestas. La tabla no sustituye las capturas que deben tomarse en terminales separadas.

**Cuadro 4.1:** Resultados de las seis sesiones cliente-servidor.

| Pr. | Proto. | Mensaje enviado | Respuesta | Resultado |
|---|---|---|---|---|
| midrule 1 | UDP | Hola servidor | Mensaje recibido: Hola servidor | Correcto |
| 2 | UDP | Prueba de mensajes | Mensaje recibido: Prueba de mensajes | Correcto |
| 3 | UDP | UDP funciona | Mensaje recibido: UDP funciona | Correcto |
| 4 | TCP | BUSCAR calculadora | Servicio encontrado en el puerto 7000 | Correcto |
| 5 | TCP | BUSCAR inventario | Servicio encontrado en el puerto 7001 | Correcto |
| 6 | TCP | BUSCAR desconocido | Servicio no encontrado | Correcto |

En cada sesión se envió también `SALIR` para cerrar el servidor. En las sesiones TCP de búsqueda exitosa, primero se registró el servicio correspondiente.

## 4.2 Preguntas de análisis

**1. ¿Qué métodos bloquean mientras esperan datos o conexiones?** En UDP, `DatagramSocket.receive()`. En TCP, `ServerSocket.accept()` y `BufferedReader.readLine()`; la apertura de la conexión puede bloquear durante el establecimiento. Leer del teclado también puede esperar entrada. Los métodos de envío no esperan una respuesta de la aplicación por sí mismos.

**2. ¿Por qué UDP no necesita `accept()`?** Porque no crea una conexión previa entre cliente y servidor. Cada datagrama contiene los datos necesarios para que el receptor identifique la dirección y el puerto de origen. En cambio, TCP establece una conexión y el servidor la recibe con `accept()`.

**3. ¿Qué diferencias existen entre `DatagramSocket`, `Socket` y `ServerSocket`?** `DatagramSocket` envía y recibe datagramas UDP independientes; `Socket` representa un extremo de una conexión TCP; y `ServerSocket` escucha un puerto y acepta nuevas conexiones TCP. TCP conserva el orden de los bytes recibidos y requiere delimitar los mensajes, aquí con saltos de línea.

**4. ¿Qué protocolo usarían para el directorio y por qué?** TCP, porque permite intercambiar varias órdenes por una conexión y proporciona entrega fiable y ordenada. Esto simplifica la implementación de un directorio que necesita recibir y contestar solicitudes correctamente. UDP requiere implementar controles adicionales si se desea garantizar entrega y reintentos.

**5. ¿Qué sucede cuando se inicia el cliente antes que el servidor?** En TCP, el intento de conexión suele fallar con `ConnectException: Connection refused` si no hay un proceso escuchando. En UDP, el envío puede finalizar sin error porque no hay establecimiento de conexión; el cliente espera una confirmación que no llegará y, en esta implementación, termina con `SocketTimeoutException` tras tres segundos.

## 4.3 Conclusión

El paso de mensajes permite colaborar a procesos independientes sin utilizar memoria compartida. UDP utiliza datagramas y no establece conexiones; TCP utiliza un flujo de bytes asociado a una conexión. En ambos ejercicios se programó un intercambio de solicitud y respuesta, con operaciones bloqueantes de recepción. Las tres pruebas UDP y las tres pruebas TCP completaron correctamente sus solicitudes. Los registros se conservan para reproducirlas; antes de entregar deberán añadirse capturas auténticas de ambas terminales.